State-Filtered Deliberation: Stage-Isolated Context Management for Small Open-Weight Software Engineering Agents

Karandeep Singh Sodhi
Independent Researcher
Submitted to the Gemma 4 Developer Agent Challenge (Paper Track) — October 2026

Abstract

Autonomous software engineering (SWE) agents built on small open-weight language models (such as 4-bit quantized Gemma 4 E2B and E4B) face severe operational constraints when executing multi-turn debugging trajectories. On the official `gemma-4-developer-agent` benchmark, while 31B submissions with fine-tuned LoRA adapters reach resolution rates of 22.0%–24.0%, all 24 participating teams on the official Small Model Prize leaderboard (E2B/E4B) score between 0.0% and 6.0% (median 3.0%). In standard single-agent loops, tool observations from file reads, graph queries, and test executions accumulate in the prompt history unless isolated or compacted.

We study **State-Filtered Deliberation (SFD)**, a declarative three-stage `SequentialAgent` architecture built on Google's Agent Development Kit (ADK) and the `swegemma` harness. SFD partitions the repair workflow into three role-restricted sub-agents—`TriageAgent` (read-only localization), `PatchAgent` (code editing), and `VerifierAgent` (targeted testing and submission)—and configures `include_contents: "none"` on `PatchAgent` and `VerifierAgent` so that intermediate tool calls and observations from preceding stages are sliced out of the prompt context at stage boundaries while passing structured text state variables (`localization_packet` and `patch_manifest`) through `session.state`. We evaluate SFD on `gemma-4-e4b-it-qat-w4a16-ct` across a deterministic stratified subset of $n = 8$ benchmark tasks spanning all four competition repositories against both the official competition starter kit (`flat_starter`, using an optional `code_analyzer` `AgentTool`) and an unconfounded single-agent baseline (`flat_controlled`, sharing identical tools, prompts, and sampling parameters). On this $n = 8$ paired sample, SFD resolves 1/8 tasks (12.5%, 95% Wilson CI [2.2%, 47.1%]) compared to 0/8 (0.0%) for `flat_starter` (McNemar $p = 1.0000$) and 0/8 (0.0%) for `flat_controlled` (McNemar $p = 1.0000$), while achieving a mean single-call peak prompt length of 9298.1 tokens (vs. 12855.2 for `flat_starter` and 12456.4 for `flat_controlled`) and resetting stage-entry prompt tokens from a mean triage peak of 8558.5 tokens down to 1534.9 tokens at `PatchAgent` entry and 1559.4 tokens at `VerifierAgent` entry. All execution traces (`ATIF-v1.7`), patches, and aggregation scripts are released for independent verification.

1. Introduction

Repository-level software engineering benchmarks such as SWE-bench ((Jimenez et al., 2024)) evaluate whether language model agents can diagnose bug reports, navigate multi-file codebases, synthesize patches, and pass unit test suites. While frontier models (100B+ parameters) and large open-weight models achieve strong resolution rates using iterative ReAct loops ((Yao et al., 2023); (Yang et al., 2024); (Wang et al., 2024)), deploying autonomous coding agents on edge hardware or local developer workstations requires compact, quantized open-weight models such as the Gemma 4 family (E2B, E4B, 12B, 31B) ((Gemma Team, Google DeepMind, 2026)).

Empirical evidence from the Google Gemma 4 Developer Agent Challenge highlights a sharp capability and reliability gap between 31B and small edge models (E2B/E4B). On the official `gemma-4-developer-agent` leaderboard (as of October 2026), the competition starter baseline running on `gemma-4-31b-it-qat-w4a16-ct` with fine-tuned LoRA adapters achieves a 22.0% (`0.22`) public resolution rate and the top overall 31B submission reaches 24.0% (`0.24`). By contrast, across all 24 participating teams on the official Small Model Prize leaderboard (restricted to `gemma-4-e4b-it-qat-w4a16-ct` and `gemma-4-e2b-it-qat-w4a16-ct`), public resolution rates range from 0.0% (`0.00`) to a ceiling of 6.0% (`0.06`), with a median score of 3.0% (`0.03`).

We investigate how agent workflow structure and context history management affect `gemma-4-e4b-it-qat-w4a16-ct` under the competition's declarative Google Agent Development Kit (ADK) and `swegemma` evaluation harness. Specifically, we formulate and test three operational hypotheses regarding small quantized coding agents:

1. **Context Growth & Signal Dilution Hypothesis:** In a continuous single-agent loop, appending raw tool outputs (file slices, call-graph queries, and command outputs) increases per-turn prompt length $|C_t|$. A possible mechanism is that dispersion of attention over growing key-value histories in 4-bit quantized weights increases repetitive reasoning loops, tool-call procrastination, or malformed edit arguments.
2. **Discretionary Sub-Agent Under-Utilization Hypothesis:** The official competition starter kit (`sample_submission/agent.yaml`) already provides a read-only `code_analyzer` sub-agent wrapped as an `AgentTool` (`skip_summarization:`

`true`) alongside all nine built-in `swegemma` tools on the root `coder` agent. However, because invoking `code_analyzer` is discretionary, we hypothesize that a small 4B model frequently bypasses the sub-agent and executes `read_file` and `run_command` directly inside its root context window.

3. **Stage-Isolated Sequential Decomposition Hypothesis:** Replacing discretionary delegation with a mandatory, role-partitioned `SequentialAgent` pipeline (`TriageAgent` $\rightarrow$ `PatchAgent` $\rightarrow$ `VerifierAgent`) that enforces `include_contents: "none"` at stage transitions deterministically bounds stage-entry prompt length and enforces separation between read-only localization, code editing, and test verification.

To test these hypotheses without confounding prompt wording, tool access, or sampling hyperparameters, we conduct a paired evaluation on `gemma-4-e4b-it-qat-w4a16-ct` across three configurations on a deterministic stratified task subset ($n = 8$): (1) the official competition starter architecture (`flat_starter`), (2) a controlled single-agent baseline (`flat_controlled`) given the exact same prompt rules and tool set as our method, and (3) **State-Filtered Deliberation** (`sfd`).

2. Problem Formulation & Context Mechanics in `swegemma` and Google ADK

2.1 Prompt Growth and the Token Signal Ratio

Consider an agent trajectory across LLM calls $t \in \{1, 2, ..., T\}$. Let P_t denote the prompt token count and C_t the completion token count (including `<|channel>thought` tokens and tool-call JSON payloads) on call t. In a standard continuous session (`include_contents: "default"`), the prompt at call t concatenates the system instruction s_0, the initial harness task prompt u_0, and all prior assistant turns a_i and tool observations o_i:

$$P_t = |s_0| + |u_0| + \sum_{i=1}^{t-1}(|a_i| + |o_i|)$$

Let C_{target} denote the minimal subset of repository code tokens required to localize and write the patch. The **Token Signal Ratio** at call t,

$$\rho_t \equiv \frac{|C_{\text{target}}|}{P_t},$$

decreases monotonically as P_t grows with exploratory file reads and shell outputs. Note that in the `swegemma` harness, even at call $t = 1$, $|s_0| + |u_0|$ is already substantial (3,800–4,100 tokens) because `build_agent_prompt()` includes the problem statement, budget rules, tool schemas, and up to 200 breadth-first directory tree entries under `/workspace`. Furthermore, `swegemma` configures Google ADK's `EventsCompactionConfig` (`compaction_interval=15`, `token_threshold=14336`, `event_retention_size=5`) to prevent 32,768-token window overflows on very long trajectories.

2.2 Exact Stage-Boundary Slicing via `include_contents: "none"` in Google ADK

Rather than waiting for 15 turns to trigger LLM compaction, SFD uses Google ADK's `SequentialAgent` combined with `include_contents: "none"` on downstream stages. Inspection of `google-adk==1.36.1` (`google/adk/flows/llm_flows/contents.py` and `functions.py`) reveals the exact runtime mechanics:

- When a sub-agent executes a tool call, `functions.py` records both the `function_call` event and the resulting `function_response` event with `event.author = invocation_context.agent.name` (e.g., `"PatchAgent"`).
- When `include_contents: "none"` is configured on `PatchAgent` (or `VerifierAgent`), `_ContentLlmRequestProcessor` invokes `_get_current_turn_contents(branch, session.events, agent.name)`. This function scans `session.events` backward from `len(events) - 1` and stops at the first event where `event.author == "user"` or `_is_other_agent_reply(agent.name, event)` is `True`—which is the **immediately preceding sub-agent's final text emission event** e_k (for `PatchAgent`) or e_m (for `VerifierAgent`).
- `_get_current_turn_contents` then slices `session.events[k:]` (or `session.events[m:]`). Consequently, all prior events `events[:k]`—including the initial 200-entry workspace layout prompt `events[0]` and every intermediate `function_call`, `function_response`, and thought block from earlier sub-agents—are excluded from `llm_request.contents`.
- Simultaneously, within the active sub-agent's own multi-step tool loop, all new tool calls and responses (`events[k+1:]`) have `event.author == agent.name`, so the backward scan passes over them and retains the sub-agent's complete within-stage tool history until that sub-agent emits its final text response.

3. Architecture & Experimental Configurations

3.1 Built-In `swegemma` Harness Capabilities and the Official Starter Kit

All three configurations execute inside the competition's `swegemma` two-phase evaluation harness. We explicitly credit `swegemma` (`swegemma.tools`) for providing the nine built-in tools available to all submissions:

- **Execution & Lifecycle Tools:** `run_command` (300s default / 60s configured timeout, 5,000-character stdout/stderr truncation), `get_status` (free budget inspection), and `submit_patch` (free `git add -N . && git diff` extraction with automatic stripping of protected test and runner config files).
- **Workspace File Tools:** `read_file` (capped at 150 lines and 10,000 characters per call), `edit_file` (3-tier exact/flexible/regex replacement with 5,000-character diff truncation), and `write_file`.
- **Pre-Computed Code Intelligence Graph Tools:** `search_similar_code` (cosine similarity search over pre-computed `.npz` symbol embeddings), `get_code_neighbors` (caller/callee lookup on the pre-computed NetworkX `CustomMultiDiGraph`), and `get_code_subgraph` (induced subgraph extraction).

The official competition starter kit (`sample_submission/agent.yaml`, evaluated here as `flat_starter`) is not a naive bash-only agent: it binds all nine built-in tools to a root `LlmAgent` (`swe_baseline_agent`) and additionally attaches a read-only `code_analyzer` sub-agent (`sub_agents/code_analyzer.yaml`) wrapped as an `AgentTool` with `skip_summarization: true`. When the root agent chooses to call `code_analyzer(request=...)`, exploration occurs in an isolated child session; however, whether `code_analyzer` is called is left entirely to the root model's tool selection.

3.2 State-Filtered Deliberation (`sfd`) Pipeline

SFD (`sub_sfd`) replaces the single root `LlmAgent` with a deterministic 3-stage `SequentialAgent` (`TriageAgent` → `PatchAgent` → `VerifierAgent`):

1. **Stage 1 (`TriageAgent`, `include_contents: "default"`, `output_key: "localization_packet"`):** Restricted in YAML config to five read-only tools (`search_similar_code`, `get_code_neighbors`, `get_code_subgraph`, `read_file`, `get_status`). It cannot modify files or run shell commands. Its prompt directs it to make at most 3 localization tool calls and output a `<localization_packet>` text block identifying `target_file`, `target_symbol`, `line_range`, `root_cause_and_fix`, and `test_command`.
2. **Stage 2 (`PatchAgent`, `include_contents: "none"`, `output_key: "patch_manifest"`):** Restricted in YAML config to four workspace tools (`read_file`, `edit_file`, `write_file`, `get_status`). At stage entry, `google-adk` slices out `TriageAgent`'s tool history and `events[0]`, and `inject_session_state()` (`google/adk/utils/instructions_utils.py`) interpolates `{problem_description?}` and `{localization_packet?}` (using optional `?` suffixes so missing keys never raise `KeyError`) into `PatchAgent`'s instruction. It applies the minimal source edit and outputs `<patch_manifest>`.
3. **Stage 3 (`VerifierAgent`, `include_contents: "none"`):** Configured with `run_command`, `read_file`, `edit_file`, `get_status`, and `submit_patch`. Its instruction interpolates `{problem_description?}`, `{localization_packet?}`, and `{patch_manifest?}`, discarding `PatchAgent`'s intermediate file-read and diff observations. It executes a targeted test via `run_command`, repairs any syntax/test regression via `edit_file`, and calls `submit_patch`.

3.3 Enforced Caps vs. Prompt-Level Directives

To avoid conflating declarative software guarantees with prompt instructions, Table 1 distinguishes properties enforced by the `swegemma` / ADK runtime from behaviors requested via system prompts.

Mechanism / Rule	Enforced in Code / YAML Config	Prompt-Requested Only (Unenforced)
Stage execution & tool separation	`SequentialAgent` order; per-stage `tools:` lists in `sub_agents/*.yaml`	Adhering to ≤ 3 tool calls in `TriageAgent` / `PatchAgent`
Cross-stage context reset	`include_contents: "none"` slices `events[k:]` at stage entry	Within-stage turn count (bounded only by global `max_turns=20`)
State packet handoff	`output_key` writes final text to `session.state`; `{var?}` interpolation	Formatting text inside `<localization_packet>` / `<patch_manifest>` tags
Observation size caps	`read_file` ≤ 150 lines / 10k chars; `run_command` stdout ≤ 5k chars	Reading narrow `start_line..end_line` slices
Test & workspace hygiene	`strip_protected_files_from_patch` strips edits to `tests/*`, `pytest.ini`, `conftest.py`	Running targeted `pytest` (not bare `pytest`); putting scratch scripts in `/tmp/`
Task session budgets	`eval_config.yaml`: `max_tool_calls=15`, `max_turns=20`, `max_time_minutes=2.5`	Calling `submit_patch` before budget exhaustion (harness has auto-diff fallback)

Table 1: Separation between runtime-enforced constraints (`swegemma` / `google-adk`) and prompt-level directives.

4. Experimental Setup & Paired Evaluation Results

4.1 Hardware, Model, and Unconfounded Experimental Design

All experiments were executed on Kaggle's 4x NVIDIA L4 GPU environment (`machine_shape: "NvidiaL4"`, 96 GB total VRAM, `tensor_parallel_size=4`, `vLLM` serving `gemma-4-e4b-it-qat-w4a16-ct` with `max_model_len=32768`, `tool_call_parser="gemma4"`, `reasoning_parser="gemma4"`). Every configuration shares the exact same generation parameters (`temperature: 0.0`, `seed: 42`, `max_output_tokens: 8192`, `thinking_budget: 1536`, `include_thoughts: true`) and task budgets (`max_tool_calls: 15`, `max_turns: 20`, `max_time_minutes: 2.5`, `timeout_seconds: 60`, `verification_timeout_seconds: 120`).

To eliminate task cherry-picking, we selected a deterministic stratified subset of $n = 8$ tasks by taking the first k_r tasks per repository in canonical `tasks.jsonl` order across all four repositories in the benchmark (`fastapi/fastapi`, `Textualize/rich`, `psf/requests`, and `encode/httpx`). On each task, we ran three configurations back-to-back against the same vLLM server:

- **`flat_starter`**: Official competition starter kit (`sample_submission/agent.yaml` with `code_analyzer AgentTool` and `prompts/system.md`, adapted to `gemma-4-e4b-it-qat-w4a16-ct`).
- **`flat_controlled`**: Single `LlmAgent` with the exact same 9 tools and combined 3-phase prompt instructions as `sfd`, isolating the effect of `SequentialAgent` + `include_contents: "none"` + stage tool partitioning.
- **`sfd`**: Our 3-stage `SequentialAgent` pipeline.

4.2 Aggregate Resolution, Patch Generation, and Context Metrics

Table 2 reports the aggregate metrics computed by `scripts/aggregate.py` directly from the `swegemma` ATIF v1.7 traces (`traces/`). **Mean Peak Prompt** denotes the mean across tasks of each task's maximum single-call `prompt_tokens` ($\max_t P_t$), while **Max Peak Prompt** is the hard maximum single-call `prompt_tokens` observed across all calls and tasks.

Configuration	Resolved ($\frac{k}{n}$)	95% Wilson CI	Non-Empty Patch	Mean Peak P_t	Max Peak P_t	Mean Cum. Tokens	Mean Tools	Mean Time
Official Starter Kit (`agent_tool` pattern)	0/8 (0.0%)	[0.0%, 32.4%]	6/8 (75.0%)	12855.2	14736	93509.9	8.9	103.2s
Controlled Single-Agent (`LlmAgent`)	0/8 (0.0%)	[0.0%, 32.4%]	8/8 (100.0%)	12456.4	14912	109507.1	9.8	89.0s
State-Filtered Deliberation (`SequentialAgent`)	1/8 (12.5%)	[2.2%, 47.1%]	7/8 (87.5%)	9298.1	14454	80790.9	10.9	92.7s

Table 2: Paired evaluation results on `gemma-4-e4b-it-qat-w4a16-ct` across $n = 8$ stratified tasks on 4x NVIDIA L4 GPUs.

Statistical Comparison of Resolution Rates:

- **`sfd` vs. `flat_starter`:** Absolute difference $\Delta = +12.50\%$, 95% paired bootstrap CI $[-0.00\%, +37.50\%]$, discordant pairs $(b = 1, c = 0)$, exact two-sided McNemar $p = 1.0000$.
- **`sfd` vs. `flat_controlled`:** Absolute difference $\Delta = +12.50\%$, 95% paired bootstrap CI $[-0.00\%, +37.50\%]$, discordant pairs $(b = 1, c = 0)$, exact two-sided McNemar $p = 1.0000$.

Measured Stage-Boundary Context Reset in `sfd`: Across the `sfd` trajectories, `TriageAgent` reaches a mean peak prompt of **8558.5 tokens** (max **14454 tokens**). When `SequentialAgent` transitions to `PatchAgent` (`include_contents: "none"`), the entry prompt on `PatchAgent`'s first LLM call drops to a mean of **1534.9 tokens** (max **2126 tokens**, a **−82.1%** reduction from `TriageAgent` peak)—empirically confirming that `_get_current_turn_contents()` slices out both `TriageAgent`'s intermediate tool observations and the initial 200-entry workspace layout tree (`events[0]`). Within `PatchAgent`, prompt tokens reach a mean peak of **5730.1 tokens** (max **12680 tokens**) before resetting again at `VerifierAgent` entry (`include_contents: "none"`) to a mean of **1559.4 tokens** (max **2165 tokens**). Across all three configurations, zero tasks hit the 32,768-token hardware overflow ceiling (`0.0%` context overflow rate) under the 15-tool-call budget.

5. Per-Task Trace Analysis & Failure Taxonomy

Table 3 lists the per-task outcome and single-call peak prompt token count ($\max_t P_t$) for every evaluated task across all three configurations, and Table 4 reports the exact stage-by-stage prompt token resets across all $n = 8$ `sfd` trajectories.

Task ID	Repository	`flat_starter` (Peak P_t)	`flat_controlled` (Peak P_t)	`sfd` (Peak P_t)
`fastapi\_15588`	`fastapi/fastapi`	Fail (test verification failed) (13083)	Fail (test verification failed) (10785)	Fail (test verification failed) (7291)
`fastapi\_15589`	`fastapi/fastapi`	Fail (test verification failed) (14711)	Fail (test verification failed) (14912)	Fail (timeout or budget no patch) (14454)
`fastapi\_15661`	`fastapi/fastapi`	Fail (test verification failed) (11099)	Fail (test verification failed) (14445)	Fail (test verification failed) (7180)
`httpx\_3672`	`encode/httpx`	Fail (test verification failed) (14736)	Fail (test verification failed) (13010)	Fail (test verification failed) (8140)
`requests\_7502`	`psf/requests`	Fail (test verification failed) (10436)	Fail (test verification failed) (11238)	Fail (test verification failed) (14446)
`requests\_7505`	`psf/requests`	Fail (test verification failed) (10800)	Fail (test verification failed) (12318)	Fail (test verification failed) (10659)
`rich\_4070`	`Textualize/rich`	Fail (timeout or budget no patch) (13298)	Fail (test verification failed) (14826)	Fail (test verification failed) (8499)
`rich\_4077`	`Textualize/rich`	Fail (timeout or budget no patch) (14679)	Fail (test verification failed) (8117)	Pass (3716)

Table 3: Task-by-task paired outcomes and single-call peak prompt tokens ($\max_t P_t$) on `gemma-4-e4b-it-qat-w4a16-ct`.

Task ID	Triage Entry	Triage Peak	Patch Entry (Δ)	Patch Peak	Verifier Entry	Verifier Peak
fastapi_15588	3373	4979	1877 (-62.3%)	7291	1818	2333
fastapi_15589	3083	14454	1534 (-89.4%)	12680	1588	1588
fastapi_15661	2993	3873	1505 (-61.1%)	7180	1557	4875
httpx_3672	2116	7842	1261 (-83.9%)	5007	1304	8140
requests_7502	1914	14446	1480 (-89.8%)	2714	1427	3042
requests_7505	2102	10659	1293 (-87.9%)	2784	1362	1812
rich_4070	4829	8499	2126 (-75.0%)	5380	2165	7542
rich_4077	2903	3716	1203 (-67.6%)	2805	1254	1970

Table 4: Measured prompt token counts (P_t) at stage entry and stage peak across all $n = 8$ `sfd` runs (`traces/eval_results/sfd/traces/`).

Rule-Based Unresolved Failure Taxonomy: Classifying every unresolved task into mutually exclusive failure buckets yields:

- **Official Starter Kit (`agent_tool` pattern)**: `timeout_or_budget_no_patch`: 2 (`rich_4070`, `rich_4077`); `test_verification_failed`: 6 (`fastapi_15588`, `fastapi_15589`, `fastapi_15661`, `httpx_3672`, `requests_7502`, `requests_7505`).
- **Controlled Single-Agent (`LlmAgent`)**: `test_verification_failed`: 8 (`fastapi_15588`, `fastapi_15589`, `fastapi_15661`, `httpx_3672`, `requests_7502`, `requests_7505`, `rich_4070`, `rich_4077`).
- **State-Filtered Deliberation (`SequentialAgent`)**: `timeout_or_budget_no_patch`: 1 (`fastapi_15589`); `test_verification_failed`: 6 (`fastapi_15588`, `fastapi_15661`, `httpx_3672`, `requests_7502`, `requests_7505`, `rich_4070`).

Trace Findings on Hypotheses 1–3 & Concrete Case Studies:

1. **Discretionary `AgentTool` Bypass in `flat_starter` (Testing Hypothesis 2):** Inspection of the ATIF v1.7 traces for `flat_starter` shows that across 8 tasks and 79 total tool invocations, `swe_baseline_agent` called `code_analyzer_agent` only **2 times (2.5% of tool calls)**, occurring on just 2/8 tasks (`requests_7505` and `rich_4077`), while **6/8 tasks (75.0%)** bypassed `code_analyzer_agent` entirely and executed all file reads and commands directly in the root context window. On both `Textualize/rich` tasks (`rich_4070` and `rich_4077`), `flat_starter` exhausted its time or tool-call budget in exploratory reads without ever calling `edit_file` (`timeout_or_budget_no_patch`, 0/2 patches generated on `Textualize/rich`).
2. **Case Study 1 — `rich_4077` (Resolved by `sfd`, Failed by `flat_starter` and `flat_controlled`):** Task `rich_4077` requires adding a missing `isatty()` method to `FileProxy` in `rich/file_proxy.py` that delegates to `self.__file.isatty()`.
   - In `flat_starter` (`trace_rich_4077.json`), the root agent spent all 15 tool calls running shell commands and reading files in its main context (`14,679` peak prompt tokens, `137,296` cumulative tokens), invoking `code_analyzer_agent` only on tool call 13 and terminating with an empty patch (`0` chars).
   - In `flat_controlled` (`trace_rich_4077.json`), the single agent localized `rich/file_proxy.py` (`8,117` peak prompt tokens, `51,036` cumulative tokens) and generated a 454-character patch, but wrote `import os` and `return os.isatty(self.__file.fileno())`, which fails Phase 2 `pytest` verification (`test_verification_failed`) when the wrapped stream does not expose an OS file descriptor.
   - In `sfd` (`trace_rich_4077.json`), `TriageAgent` read `rich/file_proxy.py` (`2,903` → `3,716` prompt tokens) and emitted a `<localization_packet>` specifying `return self.__file.isatty()`. At `PatchAgent` entry, `include_contents: "none"` reset the prompt from `3,716` to `1,203` tokens (-67.6%). `PatchAgent` read lines 1–65 of `rich/file_proxy.py`, applied `def isatty(self) -> bool: return self.__file.isatty()` (`1,990` prompt tokens, `2,805` stage peak), and emitted `<patch_manifest>`. `VerifierAgent` entered at `1,254` prompt tokens, verified `tests/test_file_proxy.py`, and called `submit_patch` (`323`-char patch, `22,695` cumulative tokens, `resolved=True`, `test_exit_code=0`).
3. **Case Study 2 — `fastapi_15589` (Runtime-Enforced Slicing vs. Unenforced Within-Stage Call Count):** On `fastapi_15589` (`trace_fastapi_15589.json`), `TriageAgent` ignored the prompt-level instruction to stop after ≤ 3 tool calls and executed 10 read-only calls (`1 search_similar_code` + `9 read_file`), reaching `14,454` prompt tokens. While `include_contents: "none"` still enforced a hard stage-entry reset at `PatchAgent` (`14,454` → `1,534` prompt tokens, −89.4%), `TriageAgent` had consumed 10 of the 15 global `max_tool_calls`. After `PatchAgent` read 6 file slices (`1,534` → `10,927` prompt tokens) and attempted `edit_file`, the harness blocked the edit (`Tool call limit reached (15)`), resulting in an empty patch (`timeout_or_budget_no_patch`). This trace directly illustrates the distinction in Table 1:

cross-stage context slicing is enforced in code by `google-adk`, whereas preventing a single stage from exhausting the global `max_tool_calls=15` budget requires per-stage code caps rather than prompt instructions alone.

4. **Localization & Semantic Edit Accuracy Bottleneck:** Across the 6 tasks where `sfd` generated a non-empty patch that failed Phase 2 verification (`fastapi_15588`, `fastapi_15661`, `httpx_3672`, `requests_7502`, `requests_7505`, `rich_4070`), trace inspection confirms that context slicing kept `PatchAgent` entry prompts between `1,261` and `2,126` tokens, but the 4-bit quantized E4B model failed verification due to incomplete multi-site edits or subtle edge-case semantics tested by `task.test_patch`.

6. Related Work

Staged vs. Monolithic SWE Agents. SWE-agent ((Yang et al., 2024)) and OpenHands ((Wang et al., 2024)) equip an LLM with an interactive bash/file environment inside a continuous trajectory. Conversely, Agentless ((Xia et al., 2024)) demonstrated that a fixed pipeline of localization followed by repair without autonomous shell loops is competitive on frontier models. SFD bridges these two paradigms within Google ADK: each stage (`TriageAgent`, `PatchAgent`, `VerifierAgent`) remains a tool-using `LlmAgent`, but stage transitions enforce hard context resets and role-specific tool permissions.

Observation Masking vs. LLM Summarization. Recent empirical work by Lindenbauer et al. ((Lindenbauer et al., 2025)) (**"The Complexity Trap"**) compares LLM-based trajectory summarization against simple environment observation masking in software engineering agents, finding that masking stale tool observations halves token costs while matching or outperforming LLM summarization on SWE-bench Verified. SFD's stage transition mechanism aligns directly with this finding: rather than invoking a separate LLM summarizer call over raw file reads, `TriageAgent` emits a compact localization packet as its natural final turn, and `include_contents: "none"` masks 100% of prior raw tool observations at the `PatchAgent` and `VerifierAgent` boundaries.

7. Limitations & Threats to Validity

1. **Sample Size and Statistical Power:** Because each full two-phase task evaluation requires isolated container setup, multi-turn GPU inference, and hermetic `pytest` verification, our paired controlled evaluation covers $n = 8$ stratified tasks (24 total runs). At $n = 8$, Wilson confidence intervals are wide and small differences in resolution rate are not statistically significant (McNemar $p > 0.05$). Full 129-task evaluation is required to detect small effect sizes ($\leq 5\%$).
2. **Irrecoverable Triage Mislocalization:** Because `PatchAgent` has `include_contents: "none"` and lacks `search_similar_code` or `run_command`, if `TriageAgent` misidentifies `target_file` within its 3-call budget, `PatchAgent` cannot re-search the repository graph.
3. **Repository Skew and Pre-Training Overlap:** In the 129-task competition dataset, 51.9% of tasks come from `fastapi/fastapi` and 37.2% from `Textualize/rich`. Furthermore, because these are public open-source repositories, historical code context may overlap with Gemma 4 pre-training corpora.

8. Conclusion & Reproducibility

We presented State-Filtered Deliberation (SFD), a declarative 3-stage `SequentialAgent` architecture for small open-weight developer agents on the `swegemma` harness, and evaluated it via real paired execution traces on `gemma-4-e4b-it-qat-w4a16-ct`. While stage-boundary context slicing via `include_contents: "none"` deterministically reduces `PatchAgent` and `VerifierAgent` entry prompts to 1535–1559 tokens and reduces mean peak prompt tokens across trajectories, overall end-to-end resolution on 4-bit E4B remains tightly constrained by fault-localization accuracy and patch semantics. All real ATIF v1.7 traces (`traces/`), aggregation scripts (`scripts/aggregate.py`), and consistency checks (`scripts/check_consistency.py`) are available at `https://github.com/KarandeepSinghSodhi/gemma-sfd-agent`.

References

Gemma Team, Google DeepMind. (2026). Gemma 4 Technical Report. *Arxiv Preprint Arxiv:2607.02770*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. *International Conference on Learning Representations (ICLR)*.

Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., & Zharov, Y. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. *Arxiv Preprint Arxiv:2508.21433*.

Wang, X., Li, B., Song, Y., Frank, F. F., & others. (2024). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *Arxiv Preprint Arxiv:2407.16741*.

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. *Arxiv Preprint Arxiv:2407.01489*.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Arxiv Preprint Arxiv:2405.15793*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.